

EXPERIMENT – 0 Introduction to Arduino

PROJECT NAME

LED Blinking System

OBJECTIVE / PROBLEM STATEMENT

The objective of this experiment is to interface an LED with Arduino UNO and make the LED blink continuously at a fixed interval.

This experiment helps in understanding:

- Digital output pins of Arduino
 - Basic Arduino programming structure
 - Use of functions in Arduino
 - Timing control using `delay()`
-

COMPONENTS USED

Component	Quantity
Arduino UNO	1
LED	1
100Ω Resistor	1
Breadboard	1
Jumper Wires	Required

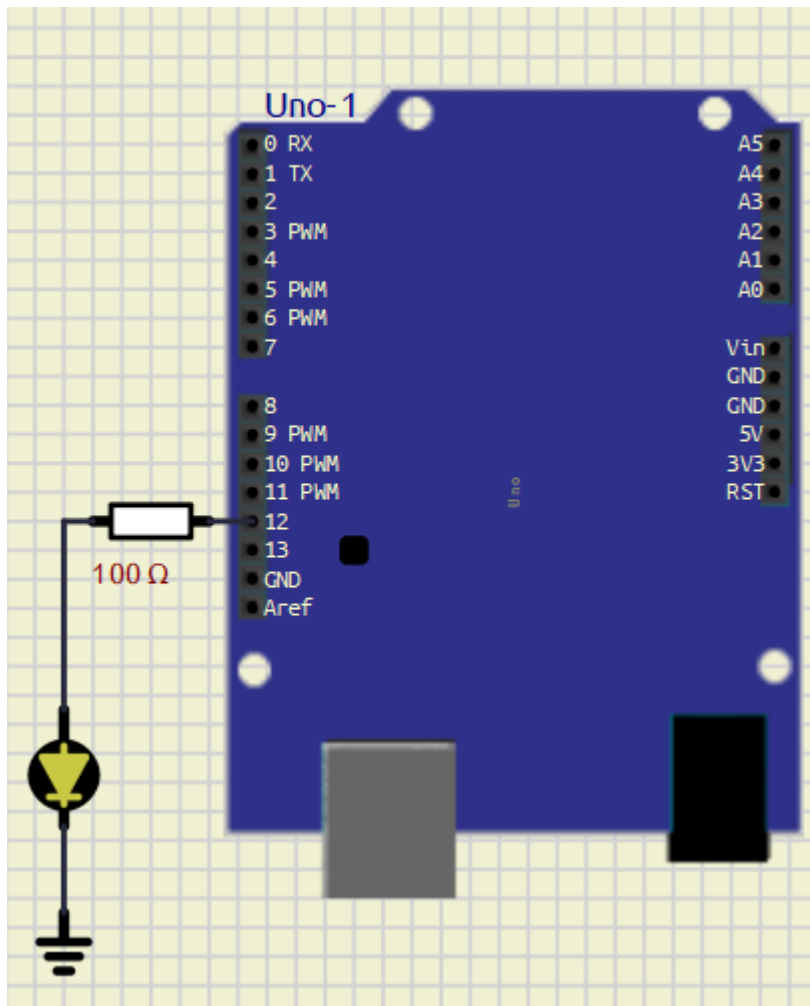
PIN CONNECTIONS

Device	Arduino Pin
LED Positive Terminal	D12
LED Negative Terminal	GND

SOFTWARE USED

- Arduino IDE
 - [SimulIDE](#)
-

CIRCUIT DIAGRAM



MAIN ARDUINO CODE

```
#define led 12 //Led Pin 12

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  toggle_led();
}

void toggle_led()
{
  digitalWrite(led, HIGH);
  delay(1000);

  digitalWrite(led, LOW);
  delay(1000);
}
```

WORKING PRINCIPLE

1. The LED is connected to digital pin D12 of Arduino UNO.
 2. In the `setup()` function, pin D12 is configured as an output pin using `pinMode()`.
 3. The `loop()` function continuously calls the `toggle_led()` function.
 4. Inside the `toggle_led()` function:
 - The LED turns ON using `digitalWrite(HIGH)`
 - Arduino waits for 1 second using `delay(1000)`
 - The LED turns OFF using `digitalWrite(LOW)`
 - Arduino again waits for 1 second
 5. This process repeats continuously, causing the LED to blink repeatedly.
-

ADVANTAGES

- Simple beginner-level Arduino project
 - Easy to understand digital output control
 - Helps in learning function-based programming
 - Useful for testing Arduino boards
-

APPLICATIONS

- Status indication systems
 - Warning indicators
 - Signal systems
 - Embedded systems learning
 - Arduino beginner projects
-

WHAT I LEARN

- Arduino UNO digital pins give approximately 5V output
 - Recommended current per digital pin is 20mA
 - Maximum current per digital pin is 40mA
 - Typical LED operating current is around 10mA to 20mA
 - LEDs should be connected using a current limiting resistor
 - Basic LED interfacing with Arduino UNO
-

CONCLUSION

The LED Blinking System using Arduino UNO was successfully implemented and tested.

The LED blinked continuously with a delay of 1 second ON and 1 second OFF. This experiment helped in understanding basic Arduino programming, digital output control, and hardware interfacing.

REFERENCES

1. Arduino Official Website
<https://www.arduino.cc/>
2. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
3. Arduino Tutorials
<https://www.arduino.cc/en/Tutorial/HomePage>
4. Arduino IDE Documentation
<https://docs.arduino.cc/software/ide/>
5. SimulIDE
<https://simulide.com/>

EXPERIMENT – 0 GO FURTHER – 2

PROJECT NAME

7 Segment Display Interfacing with Arduino UNO

OBJECTIVE / PROBLEM STATEMENT

The objective of this project is to interface a common cathode 7-segment display with Arduino UNO and display numbers sequentially.

This experiment helps in understanding:

- 7-segment display interfacing
- Digital output control
- Number display logic
- Multi-pin control using Arduino

COMPONENTS USED

Component	Quantity
Arduino UNO	1
7 Segment Display (Common Cathode)	1
220Ω Resistors	7
Jumper Wires	Required
Breadboard	1

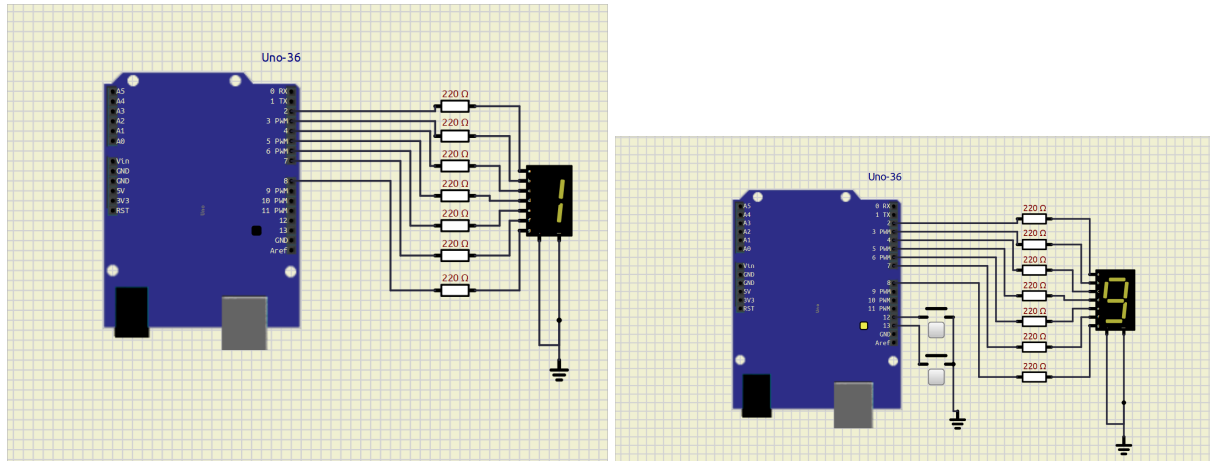
PIN CONNECTIONS

7 Segment Pin	Arduino Pin
a	D2
b	D3
c	D4
d	D5
e	D6
f	D7
g	D8

SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



MAIN ARDUINO CODE

```
// 7 Segment Display with Arduino UNO
// Common Cathode Display
```

```
int a = 2;
int b = 3;
int c = 4;
int d = 5;
int e = 6;
int f = 7;
int g = 8;
```

```
void setup() {
```

```
  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
  pinMode(d, OUTPUT);
  pinMode(e, OUTPUT);
  pinMode(f, OUTPUT);
  pinMode(g, OUTPUT);
```

```
}
```

```
void loop() {
```

```
  // Display 0
  digitalWrite(a, HIGH);
  digitalWrite(b, HIGH);
```

```
digitalWrite(c, HIGH);
digitalWrite(d, HIGH);
digitalWrite(e, HIGH);
digitalWrite(f, HIGH);
digitalWrite(g, LOW);

delay(1000);

// Display 1
digitalWrite(a, LOW);
digitalWrite(b, HIGH);
digitalWrite(c, HIGH);
digitalWrite(d, LOW);
digitalWrite(e, LOW);
digitalWrite(f, LOW);
digitalWrite(g, LOW);

delay(1000);

// Display 2
digitalWrite(a, HIGH);
digitalWrite(b, HIGH);
digitalWrite(c, LOW);
digitalWrite(d, HIGH);
digitalWrite(e, HIGH);
digitalWrite(f, LOW);
digitalWrite(g, HIGH);

delay(1000);

// Display 3
digitalWrite(a, HIGH);
digitalWrite(b, HIGH);
digitalWrite(c, HIGH);
digitalWrite(d, HIGH);
digitalWrite(e, LOW);
digitalWrite(f, LOW);
digitalWrite(g, HIGH);

delay(1000);

}
```

WORKING PRINCIPLE

1. The 7-segment display segments are connected to Arduino digital pins D2 to D8.
2. Each segment is controlled individually using `digitalWrite()`.
3. Depending on which segments are turned ON or OFF, different numbers are displayed.
4. The Arduino sequentially displays:

- 0
- 1
- 2
- 3

5. Each number remains visible for 1 second using `delay(1000)`.

6. The sequence repeats continuously.

WHAT I LEARN

- Interfacing a 7-segment display with Arduino UNO
 - Common cathode display working principle
 - Using multiple digital output pins
 - Number display logic using segments
 - Arduino digital output voltage is approximately 5V
 - Recommended current per pin is 20mA
 - Typical LED segment current is around 10mA to 20mA
-

ADVANTAGES

- Simple number display system
 - Easy to understand segment logic
 - Useful for beginners
 - Low-cost implementation
-

APPLICATIONS

- Digital counters
 - Timer displays
 - Calculator displays
 - Embedded display systems
 - Electronic meters
-

CONCLUSION

The 7-segment display was successfully interfaced with Arduino UNO.

The display sequentially showed numbers from 0 to 3 by controlling individual segments through Arduino digital pins. This project helped in understanding display interfacing and segment control logic.

REFERENCES

1. Arduino Official Website
<https://www.arduino.cc/>

2. Arduino UNO Documentation

<https://docs.arduino.cc/hardware/uno-rev3/>

3. 7 Segment Display Basics

<https://www.electronics-tutorials.ws/blog/7-segment-display-tutorial.html>

4. Arduino Tutorials

<https://www.arduino.cc/en/Tutorial/HomePage>

EXPERIMENT – 0 GO FURTHER – 3

PROJECT NAME

LED Matrix Emoji Display System

OBJECTIVE / PROBLEM STATEMENT

The objective of this project is to interface an 8×8 LED matrix with Arduino UNO and display different emoji patterns.

This experiment helps in understanding:

- LED matrix interfacing
- Row and column scanning
- Pattern generation
- Multiplexing technique

COMPONENTS USED

Component	Quantity
Arduino UNO	1
8×8 LED Matrix	1
125Ω Resistors	8
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

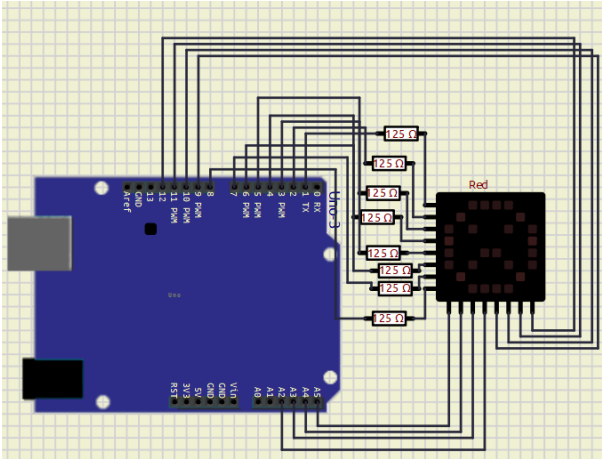
LED Matrix	Arduino Pin
Row 1	D1
Row 2	D2
Row 3	D3
Row 4	D4
Row 5	D5
Row 6	D6
Row 7	D7

LED Matrix	Arduino Pin
Row 8	D8
Column 1	A5
Column 2	A4
Column 3	A3
Column 4	A2
Column 5	D9
Column 6	D10
Column 7	D11
Column 8	D12

SOFTWARE USED

- Arduino IDE
- Proteus / SimulIDE

CIRCUIT DIAGRAM



MAIN ARDUINO CODE

```
byte rows[8] = {1,2,3,4,5,6,7,8};
byte cols[8] = {A5,A4,A3,A2,9,10,11,12};

// Emoji Patterns
byte emoji[3][8] = {

// Smiley Face
{
    B001111100,
```

```
    B01000010,
    B10100101,
    B10000001,
    B10100101,
    B10011001,
    B01000010,
    B00111100
  },

  // Sad Face
  {
    B00111100,
    B01000010,
    B10100101,
    B10000001,
    B10011001,
    B10100101,
    B01000010,
    B00111100
  }
};

void setup() {

  for(int i=0; i<8; i++) {
    pinMode(rows[i], OUTPUT);
    pinMode(cols[i], OUTPUT);
  }
}

void displayEmoji(int num) {

  for(int t=0; t<250; t++) {

    for(int r=0; r<8; r++) {

      // OFF all rows
      for(int i=0; i<8; i++) {
        digitalWrite(rows[i], LOW);
      }

      // OFF all columns
      for(int i=0; i<8; i++) {
        digitalWrite(cols[i], HIGH);
      }

      // Activate current row
      digitalWrite(rows[r], HIGH);

      // Display pattern
      for(int c=0; c<8; c++) {

        if(bitRead(emoji[num][r], 7-c)) {
          digitalWrite(cols[c], LOW);
        }
      }
    }
  }
}
```

```
    }  
  }  
  
  delay(2);  
}  
}  
}  
  
void loop() {  
  
  displayEmoji(0); // Smiley  
  //displayEmoji(1); // Heart  
  displayEmoji(2); // Sad  
}
```

WORKING PRINCIPLE

1. The 8×8 LED matrix is connected to Arduino UNO using row and column connections.
2. Arduino scans rows and columns rapidly using multiplexing.
3. Binary patterns are stored inside the `emoji` array.
4. The `displayEmoji()` function activates rows one by one and controls columns according to the stored pattern.
5. Different emoji patterns are displayed continuously on the LED matrix.
6. Due to fast scanning, the complete image appears continuously visible.

WHAT I LEARN

- Interfacing an LED matrix with Arduino UNO
- Multiplexing technique
- Row and column scanning
- Pattern generation using binary values
- Arduino digital output voltage is approximately 5V
- Recommended current per pin is 20mA
- LED matrix LEDs require current limiting resistors

ADVANTAGES

- Can display multiple patterns
- Low-cost display system
- Useful for learning multiplexing
- Simple animation implementation

APPLICATIONS

- LED display boards
 - Emoji displays
 - Scrolling text systems
 - Digital signboards
 - Embedded display projects
-

CONCLUSION

The LED matrix was successfully interfaced with Arduino UNO.

Different emoji patterns were displayed by controlling rows and columns using multiplexing techniques. This project helped in understanding matrix displays and pattern generation in embedded systems.

REFERENCES

1. Arduino Official Website
<https://www.arduino.cc/>
2. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
3. LED Matrix Basics
<https://www.electronicsforu.com/resources/learn-electronics/8x8-led-matrix-module>
4. Arduino Tutorials
<https://www.arduino.cc/en/Tutorial/HomePage>

EXPERIMENT – 1 INTERFACING WITH ARDUINO

PROJECT NAME

Digital Temperature Monitoring System

OBJECTIVE / PROBLEM STATEMENT

The main objective of this project is to design a smart temperature monitoring system using Arduino UNO and a DHT11 temperature sensor.

The system continuously measures room temperature and displays the live temperature value on an SSD1306 OLED display. According to the temperature value, the system automatically controls cooling and heating devices such as a fan and heater.

A push button is also provided to turn the complete system ON or OFF manually.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
SSD1306 OLED Display	1
Red LED	1
Green LED	1
DC Fan / Motor	1
Heater / Bulb	1
Push Button	1
Jumper Wires	Required

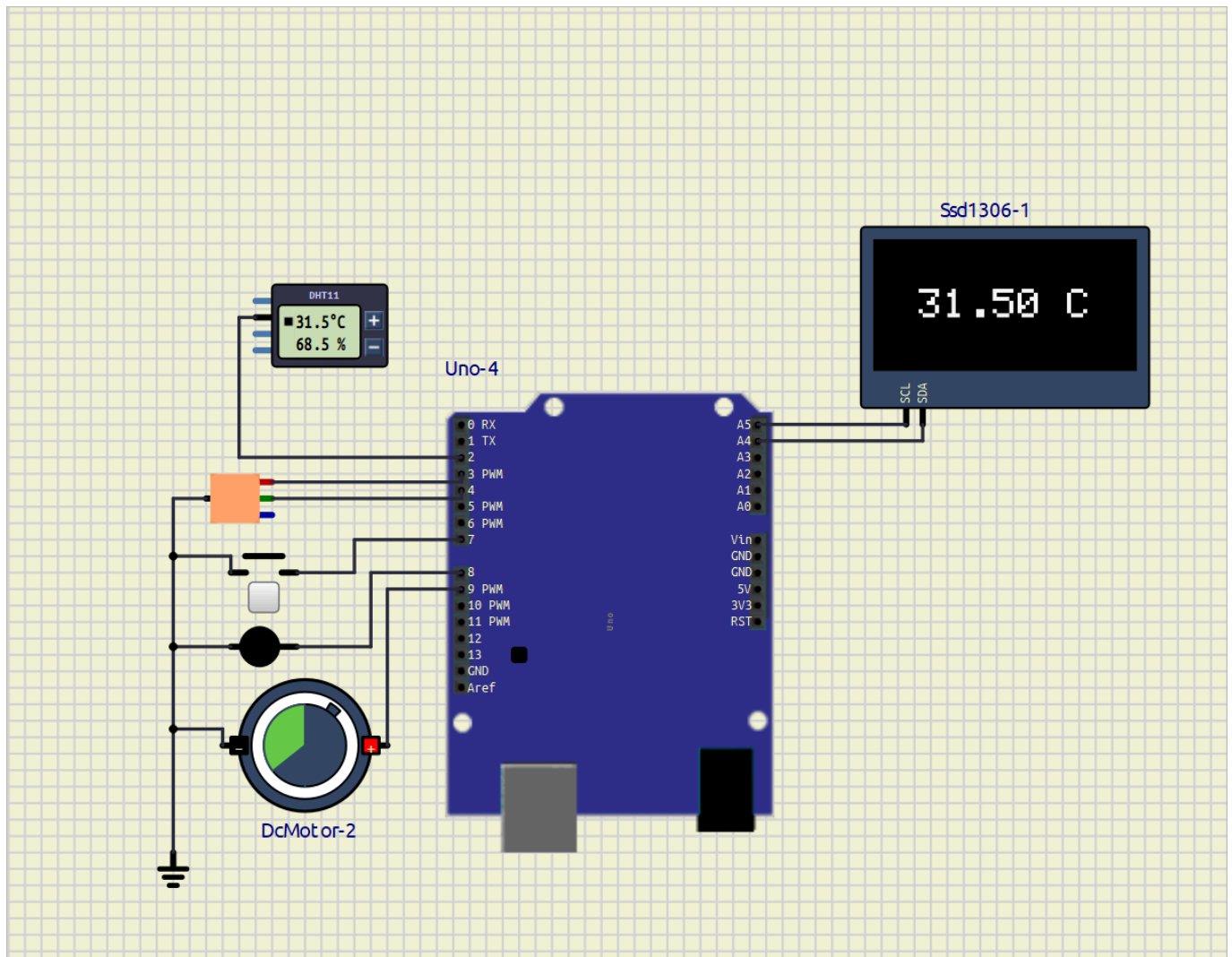
PIN CONNECTIONS

Device	Arduino Pin
DHT11 Data Pin	D2
Red LED	D3
Green LED	D4
Push Button	D7
Heater	D8
Fan	D9
OLED SDA	A4
OLED SCL	A5

SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
```

MAIN ARDUINO CODE

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define DHTPIN 2
#define DHTTYPE DHT11
```



```
DHT dht(DHTPIN, DHTTYPE);

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

bool systemON = false;

void setup()
{
  dht.begin();

  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  display.clearDisplay();

  pinMode(3, OUTPUT);

  pinMode(4, OUTPUT);

  pinMode(9, OUTPUT);

  pinMode(8, OUTPUT);

  pinMode(7, INPUT_PULLUP);
}

void loop()
{
  if(digitalRead(7) == LOW)
  {
    systemON = !systemON;

    delay(300);
  }

  if(systemON)
  {
    float temp = dht.readTemperature();

    display.clearDisplay();

    display.setTextSize(2);

    display.setTextColor(WHITE);

    String text = String(temp) + " C";

    int x = (128 - (text.length() * 12)) / 2;
```

```
int y = (64 - 16) / 2;

display.setCursor(x, y);

display.print(text);

display.display();

if(temp >= 28)
{
    digitalWrite(3, HIGH);

    digitalWrite(4, LOW);

    digitalWrite(9, HIGH);

    digitalWrite(8, LOW);
}
else
{
    digitalWrite(3, LOW);

    digitalWrite(4, HIGH);

    digitalWrite(9, LOW);

    digitalWrite(8, HIGH);
}
}

else
{
    digitalWrite(3, LOW);

    digitalWrite(4, LOW);

    digitalWrite(9, LOW);

    digitalWrite(8, LOW);

    display.clearDisplay();

    display.display();
}

delay(1000);
}
```

WORKING PRINCIPLE

1. The push button is used to turn the complete system ON or OFF.

2. When the system is ON, the DHT11 sensor continuously measures room temperature.
 3. The measured temperature value is displayed on the OLED screen in real time.
 4. The Arduino UNO compares the measured temperature with the predefined threshold temperature value.
 5. If the temperature becomes greater than or equal to 28°C:
 - Red LED turns ON
 - Fan starts running automatically
 - Heater turns OFF
 6. If the temperature becomes lower than 28°C:
 - Green LED turns ON
 - Heater turns ON automatically
 - Fan turns OFF
 7. The OLED display continuously updates the live temperature reading.
 8. When the push button is pressed again:
 - The complete system turns OFF
 - OLED display becomes blank
 - LEDs turn OFF
 - Fan and heater stop working
-

ADVANTAGES

- Automatic temperature control
 - Real-time temperature monitoring
 - Easy ON/OFF operation
 - Low-cost implementation
 - User-friendly system
 - Useful for smart automation
-

APPLICATIONS

- Smart room temperature control
 - Automatic cooling systems
 - Automatic heating systems
 - Industrial temperature monitoring
 - Home automation projects
 - Embedded systems learning
-

CONCLUSION

The Digital Temperature Monitoring System using Arduino UNO and DHT11 sensor was successfully implemented and tested in SimulIDE.

The system continuously monitored temperature and automatically controlled the fan and heater according to the temperature value. The OLED display successfully showed live temperature readings, and the push button provided convenient ON/OFF control of the complete system.

This project helped in understanding:

- Arduino interfacing
- Sensor interfacing
- OLED display handling
- Embedded system automation
- Temperature-based control systems

REFERENCES

1. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
2. DHT11 Sensor Documentation
<https://components101.com/sensors/dht11-temperature-sensor>
3. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
4. Adafruit GFX Library
<https://github.com/adafruit/Adafruit-GFX-Library>
5. Arduino DHT Sensor Library
<https://docs.arduino.cc/libraries/dht-sensor-library/>
6. Arduino UNO Pinout
<https://deepbluembedded.com/arduino-uno-pinout/>

EXPERIMENT – 1 INTERFACING WITH ARDUINO

PROJECT NAME

Temperature Monitoring-Emergency Alert System

OBJECTIVE / PROBLEM STATEMENT

This project monitors temperature using a DHT11 sensor and activates a buzzer when the temperature exceeds the threshold value of 50°C. The system is designed using Arduino UNO in SimulIDE.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
Buzzer	1
Jumper Wires	Required

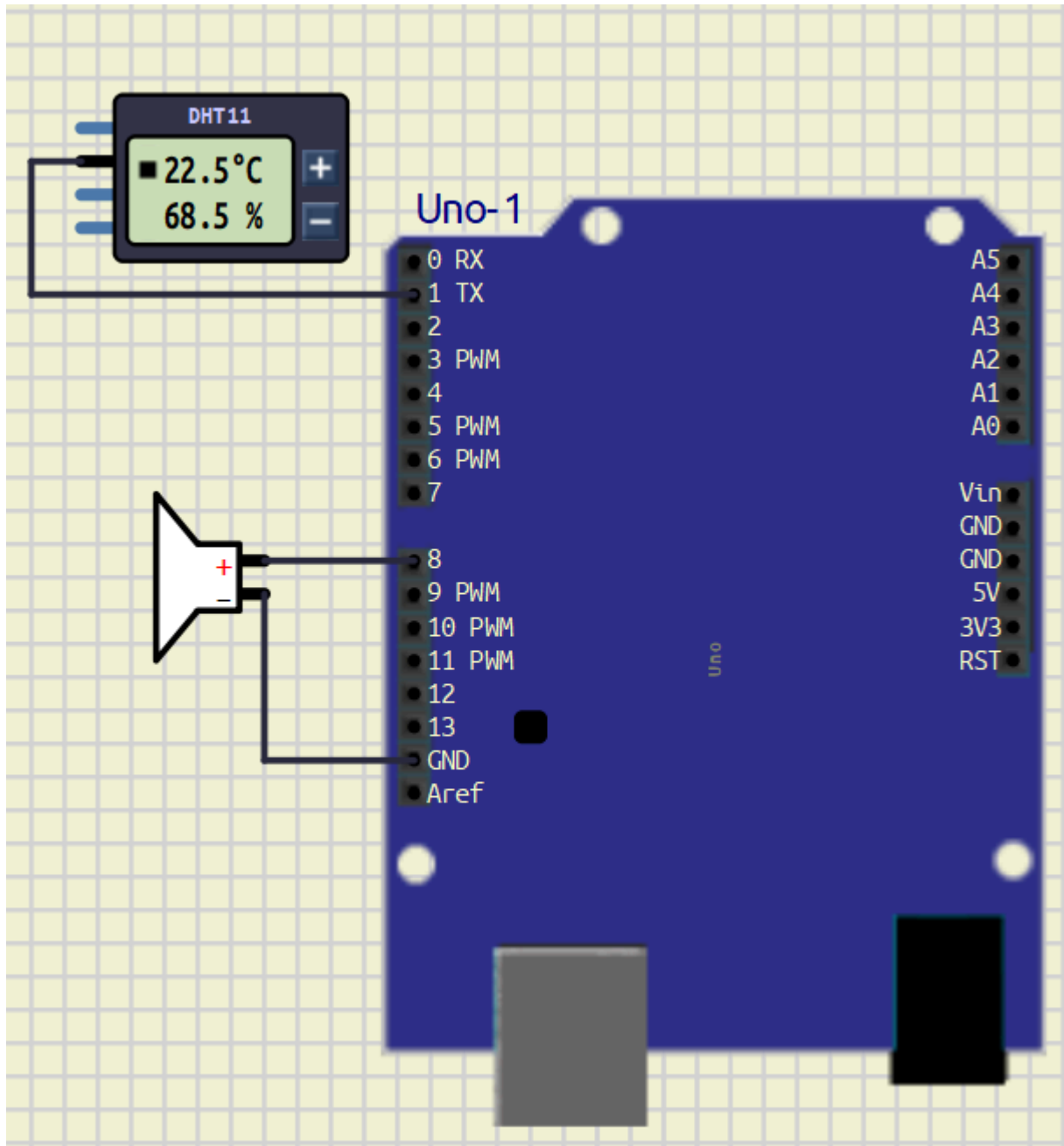
PIN CONNECTIONS

Device	Arduino Pin
DHT11 Data Pin	D1
Buzzer	D8
DHT11 VCC	5V
DHT11 GND	GND

SOFTWARE USED

- Arduino IDE
 - SimulIDE
-

Circuit diagram



REQUIRED LIBRARY

```
#include <DHT.h>
```

MAIN ARDUINO CODE

Code

```
#include <DHT.h>

#define DHTPIN 1
#define DHTTYPE DHT11
#define BUZZER 8
#define THRESHOLD 50
```

```
DHT dht(DHTPIN, DHTTYPE);

void setup()
{
    pinMode(BUZZER, OUTPUT);
    dht.begin();
}

void loop()
{
    float temp = dht.readTemperature();

    if(temp >= THRESHOLD)
    {
        digitalWrite(BUZZER, HIGH);
    }
    else
    {
        digitalWrite(BUZZER, LOW);
    }

    delay(1000);
}
```

WORKING PRINCIPLE

1. DHT11 sensor continuously reads temperature.
2. Arduino UNO compares the temperature with the threshold value.
3. If temperature is greater than or equal to 50°C:
 - Buzzer turns ON.
4. If temperature is below 50°C:
 - Buzzer remains OFF.
5. The process repeats continuously.

OUTPUT

- Buzzer remains OFF below 50°C.
- Buzzer turns ON at or above 50°C.
- System performs real-time temperature monitoring.

ADVANTAGES

- Simple and low-cost system
- Easy to implement
- Real-time monitoring
- Useful for safety applications

APPLICATIONS

- Fire warning systems
 - Temperature alert systems
 - Industrial monitoring
 - Home safety projects
 - Embedded system learning
-

CONCLUSION

The Digital Temperature Monitoring System using Arduino UNO and DHT11 sensor was successfully implemented in SimulIDE. The buzzer activated whenever the temperature crossed the threshold value of 50°C.

References

1. Arduino Project hub- With DHT11 sensor <https://projecthub.arduino.cc/arcaegecengiz/using-dht11-12f621>
2. DHT sensor Library - <https://docs.arduino.cc/libraries/dht-sensor-library/#Releases>
3. Arduino Uno board Pin Out - <https://deepbluembedded.com/arduino-uno-pinout/>

EXPERIMENT – 1 Interfacing With Arduino

PROJECT NAME

Weather Forecasting System with Extreme Temperature Indicator

OBJECTIVE / PROBLEM STATEMENT

This project monitors environmental temperature and humidity using a DHT11 sensor and predicts weather conditions based on temperature values.

The system uses RGB LED indicators to show:

- Extreme Hot Weather
- Normal Weather
- Cold Weather

A DS1307 RTC module provides real-time clock information, and an SSD1306 OLED display shows:

- Current Time
- Temperature
- Humidity
- Weather Status

A push button is also used as a master power control for turning the complete system ON and OFF.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
DHT11 Sensor	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
RGB LED	1
Push Button	1
Resistors (100Ω)	3
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

Device	Arduino Pin
DHT11 Data Pin	D7
Red LED	D2
Green LED	D3
Blue LED	D5
Power Button	D8
OLED SDA	A4
OLED SCL	A5
RTC SDA	A4
RTC SCL	A5

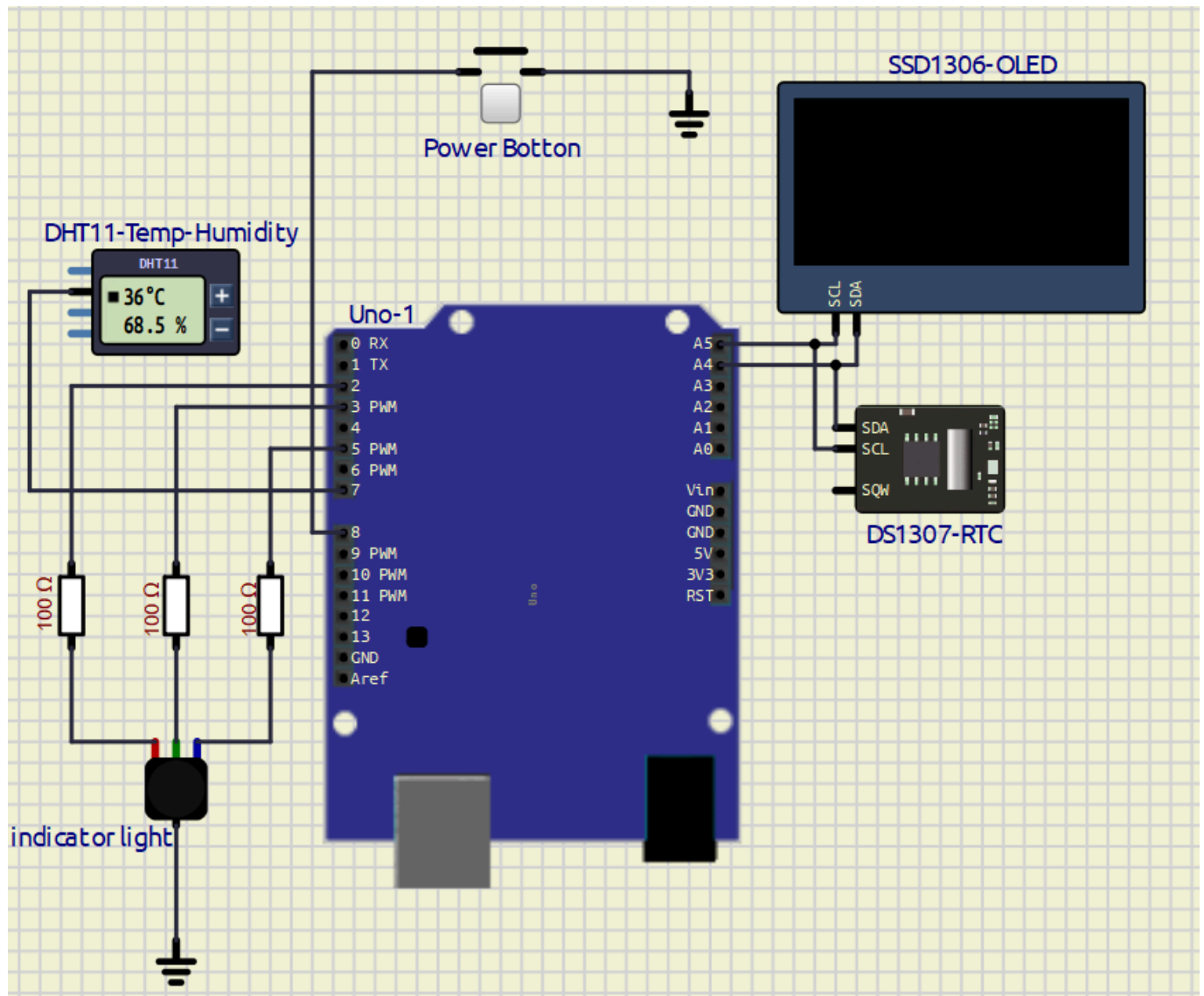
RGB LED CONNECTION

RGB Pin	Connection
Red Pin	D2 through 100Ω
Green Pin	D3 through 100Ω
Blue Pin	D5 through 100Ω
Common Cathode	GND

SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



REQUIRED LIBRARIES

```
#include <Wire.h>
#include <RTClib.h>
#include <DHT.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

MAIN ARDUINO CODE

Code

```
#include <Wire.h>
#include <RTClib.h>

#include <DHT.h>
```

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

RTC_DS1307 rtc;

#define DHTPIN 7
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

#define RED_LED      2
#define GREEN_LED    3
#define BLUE_LED     5

#define POWER_BUTTON 8

bool systemState = false;

bool lastButtonState = HIGH;

void setup()
{
  pinMode(RED_LED, OUTPUT);

  pinMode(GREEN_LED, OUTPUT);

  pinMode(BLUE_LED, OUTPUT);

  pinMode(POWER_BUTTON, INPUT_PULLUP);

  Wire.begin();

  rtc.begin();

  dht.begin();

  // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
  {
    while (1);
  }

  display.clearDisplay();
```

```
    display.display();
}

void loop()
{
    bool buttonState = digitalRead(POWER_BUTTON);

    if (
        buttonState == LOW &&
        lastButtonState == HIGH
    )
    {
        delay(50);

        systemState = !systemState;

        if (systemState == true)
        {
            display.clearDisplay();

            display.setTextSize(2);

            display.setTextColor(WHITE);

            display.setCursor(10, 25);

            display.println("POWER ON");

            display.display();

            delay(1000);
        }

        else
        {
            display.clearDisplay();

            display.setTextSize(2);

            display.setTextColor(WHITE);

            display.setCursor(10, 25);

            display.println("POWER OFF");

            display.display();

            delay(1000);

            display.clearDisplay();

            display.display();
        }
    }
}
```

```
}

lastButtonState = buttonState;

if (systemState == false)
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, LOW);

    return;
}

DateTime now = rtc.now();

float temperature = dht.readTemperature();

float humidity = dht.readHumidity();

String statusText;

if (temperature > 35)
{
    digitalWrite(RED_LED, HIGH);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, LOW);

    statusText = "EXTREME HOT";
}

else if (temperature < 20)
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, LOW);

    digitalWrite(BLUE_LED, HIGH);

    statusText = "COLD";
}

else
{
    digitalWrite(RED_LED, LOW);

    digitalWrite(GREEN_LED, HIGH);

    digitalWrite(BLUE_LED, LOW);

    statusText = "NORMAL";
}
```

```
}

display.clearDisplay();

display.setTextSize(1);

display.setTextColor(WHITE);

display.setCursor(0, 0);

display.println("WEATHER SYSTEM");

display.setCursor(0, 15);

display.print("TIME : ");

print2digit(now.hour());

display.print(":");

print2digit(now.minute());

display.print(":");

print2digit(now.second());

display.setCursor(0, 30);

display.print("TEMP : ");

display.print(temperature);

display.println(" C");

display.setCursor(0, 42);

display.print("HUM : ");

display.print(humidity);

display.println(" %");

display.setCursor(0, 55);

display.print("STATUS:");

display.println(statusText);

display.display();

delay(50);
}
```

```
void print2digit(int number)
```

```
{
  if (number < 10)
  {
    display.print("0");
  }

  display.print(number);
}
```

WORKING PRINCIPLE

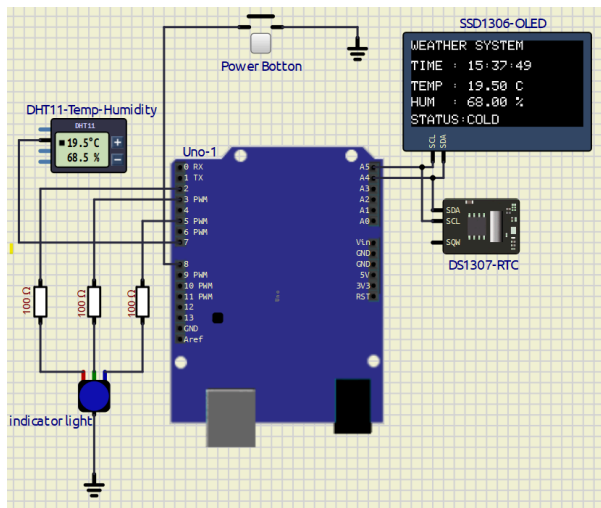
1. The system starts with a blank OLED screen.
2. Pressing the power button:
 - Turns the system ON
 - Displays "POWER ON"
3. DHT11 sensor continuously reads:
 - Temperature
 - Humidity
4. DS1307 RTC module provides:
 - Real-time clock
5. OLED displays:
 - Time
 - Temperature
 - Humidity
 - Weather Status
6. RGB LED indicates weather condition:
 - Red LED → Extreme Hot
 - Green LED → Normal Weather
 - Blue LED → Cold Weather
7. Pressing the power button again:
 - Displays "POWER OFF"
 - Turns OFF all LEDs
 - OLED returns to black screen

TEMPERATURE INDICATOR LOGIC

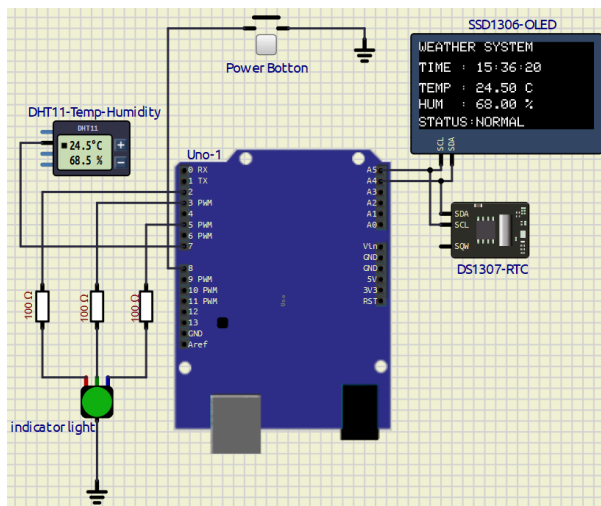
Temperature	LED Status	Weather
> 35°C	Red LED ON	Extreme Hot
20°C – 35°C	Green LED ON	Normal
< 20°C	Blue LED ON	Cold

OUTPUT

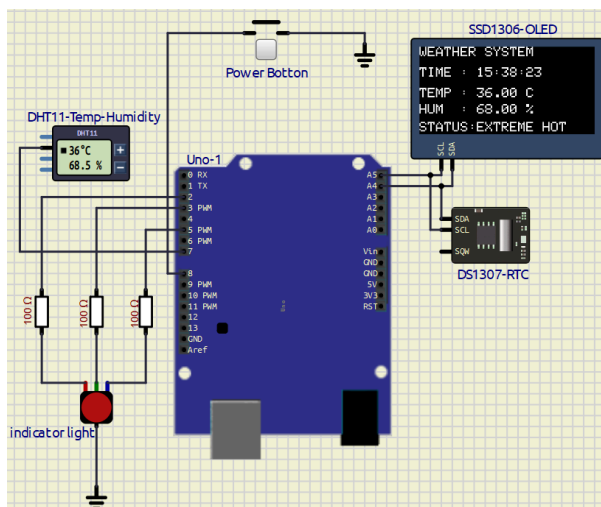
System - COLD (Temp < 20)



System - Normal (Temp > 20 & < 36)



System - Extream Hot (Temp > 35)



ADVANTAGES

- Real-time weather monitoring
- Simple embedded system design

- RGB visual indication
 - RTC-based real-time display
 - Easy ON/OFF control
 - Low-cost implementation
 - User-friendly interface
-

APPLICATIONS

- Smart weather stations
 - Environmental monitoring
 - Temperature warning systems
 - Industrial monitoring
-

CONCLUSION

The Weather Forecasting System using Arduino UNO, DHT11 sensor, DS1307 RTC module, SSD1306 OLED display, RGB LED, and push-button control was successfully implemented in SimulIDE.

The system correctly monitors weather conditions, displays real-time environmental information, and visually indicates temperature conditions using RGB LEDs.

During this experiment, I learned:

- Sensor, OLED & RTC interfacing
 - RGB LED control
 - Push-button logic
-

REFERENCES

1. Arduino UNO Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
 2. DHT11 Sensor Documentation
<https://components101.com/sensors/dht11-temperature-sensor>
 3. DS1307 RTC Datasheet
<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
 4. Adafruit SSD1306 Library
https://github.com/adafruit/Adafruit_SSD1306
 5. Adafruit GFX Library
<https://github.com/adafruit/Adafruit-GFX-Library>
 6. RTCLib Documentation
<https://github.com/adafruit/RTCLib>
-

EXPERIMENT– 2 Calculator

PROJECT NAME

Simple Calculator Using Arduino UNO, 4x4 Keypad and SSD1306 OLED Display

OBJECTIVE / PROBLEM STATEMENT

This project implements a simple calculator using Arduino UNO, a 4x4 keypad, and an SSD1306 OLED display. The calculator performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The result is displayed on the OLED screen.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
4x4 Keypad	1
SSD1306 OLED Display	1
Jumper Wires	Required
Breadboard	1

PIN CONNECTIONS

4x4 KEYPAD CONNECTIONS

Keypad Pin	Arduino Pin
R1	D1
R2	D2
R3	D3
R4	D4
C1	D5
C2	D6
C3	D7
C4	D8

Symbols mapped operators

Symbol	Operation
A	+
B	-
C	*
D	/
#	AC

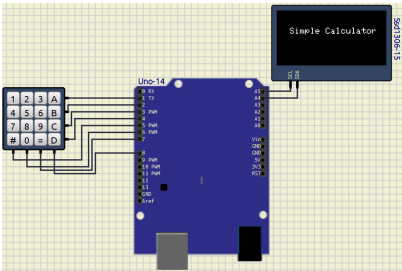
SSD1306 OLED DISPLAY CONNECTIONS

OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

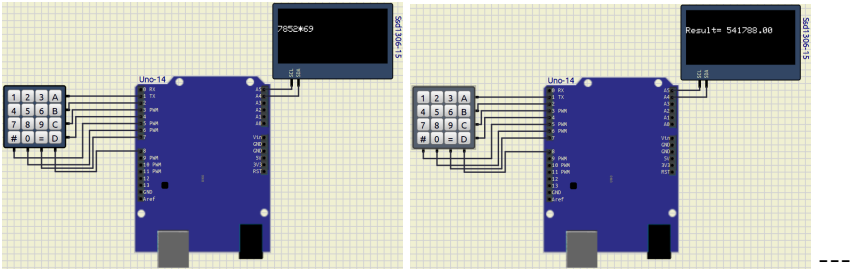
SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



Example:



\newpage

REQUIRED LIBRARIES

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Keypad.h>
```

MAIN ARDUINO CODE

Code With Comments ---> [Click here](#)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Keypad.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1', '2', '3', '+'},
  {'4', '5', '6', '-'},
  {'7', '8', '9', '*'},
  {'#', '0', '=', '/'}
};

byte rowPins[ROWS] = {1, 2, 3, 4};
byte colPins[COLS] = {5, 6, 7, 8};

Keypad keypad = Keypad(makeKeymap(keys),
                        rowPins,
                        colPins,
                        ROWS,
                        COLS);

String num1 = "";
String num2 = "";

char op;

float result;

bool operatorPressed = false;

String expression = "";
```

```
void setup()
{
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

    display.clearDisplay();

    display.setTextSize(1);

    display.setCursor(0, 0);
    display.println("Simple Calculator");

    display.setCursor(0, 20);
    display.println("Can Perform");

    display.setCursor(0, 40);
    display.println("(+, -, *, /)");

    display.display();

    delay(2000);

    display.clearDisplay();
    display.display();
}

void loop()
{
    char key = keypad.getKey();

    if(key)
    {
        // CLEAR ALL
        if(key == '#')
        {
            num1 = "";
            num2 = "";
            expression = "";

            operatorPressed = false;

            result = 0;

            display.clearDisplay();
            display.display();
        }

        // NUMBER INPUT
        else if(isDigit(key))
        {
            if(!operatorPressed)
            {
                num1 += key;
            }
            else
            {
                num2 += key;
            }
        }
    }
}
```

```
        {
            num2 += key;
        }

        expression += key;

        updateDisplay(expression);
    }

    // OPERATOR INPUT
    else if(key == '+' || key == '-' ||
            key == '*' || key == '/')
    {
        if(!operatorPressed && num1 != "")
        {
            op = key;

            operatorPressed = true;

            expression += key;

            updateDisplay(expression);
        }
    }

    // RESULT
    else if(key == '=')
    {
        float n1 = num1.toFloat();
        float n2 = num2.toFloat();

        switch(op)
        {
            case '+':
                result = n1 + n2;
                break;

            case '-':
                result = n1 - n2;
                break;

            case '*':
                result = n1 * n2;
                break;

            case '/':

                if(n2 != 0)
                {
                    result = n1 / n2;
                }
                else
                {
                    display.clearDisplay();
                }
            }
        }
    }
}
```

```
        display.setTextSize(2);

        display.setCursor(0, 20);

        display.println("ERROR");

        display.display();

        delay(2000);

        return;
    }

    break;
}

expression = "Result= " + String(result);

updateDisplay(expression);

num1 = String(result);

num2 = "";

operatorPressed = false;

expression = String(result);
    }
}

void updateDisplay(String text)
{
    display.clearDisplay();

    display.setTextSize(2);

    display.setCursor(0, 20);

    display.println(text);

    display.display();
}
```

WORKING PRINCIPLE

1. The 4x4 keypad is used to enter numbers and operators.
2. Arduino UNO continuously scans the keypad.
3. Entered values are stored as operands.
4. The selected operator determines the operation to be performed.

5. When '=' is pressed:
 - Arduino performs the calculation.
 - Result is displayed on the SSD1306 OLED display.
 6. '#' key clears all previous inputs.
 7. The process repeats continuously.
-

OUTPUT

Startup Screen

```
Simple Calculator
```

Example Calculation

```
5+3
```

```
Result= 8
```

FEATURES

- Performs Addition
 - Performs Subtraction
 - Performs Multiplication
 - Performs Division
 - OLED Display Output
 - Clear Function using '#'
 - Real-time Input Detection
 - Divide by Zero Error Handling
-

ADVANTAGES

- Simple and easy to implement
 - Low-cost embedded project
 - Real-time calculations
 - User-friendly interface
 - Useful for learning keypad interfacing
-

APPLICATIONS

- Basic calculator systems
- Embedded system projects

- Learning Arduino interfacing
 - Educational mini-projects
 - Digital input-output systems
-

CONCLUSION

The Simple Calculator using Arduino UNO, 4x4 Keypad, and SSD1306 OLED Display was successfully implemented and tested. The system successfully performs basic arithmetic operations and displays the result on the OLED screen. During the experiment I faces problem in mapping those keys and assign them their values. I overcome it by the open source resources given in web and arduino forum

REFERENCES

1. Arduino Keypad Library Documentation
<https://www.arduino.cc/reference/en/libraries/keypad/>
2. SSD1306 OLED Display Library
https://github.com/adafruit/Adafruit_SSD1306
3. Arduino UNO Official Documentation
<https://docs.arduino.cc/hardware/uno-rev3/>
4. Adafruit GFX Graphics Library
<https://github.com/adafruit/Adafruit-GFX-Library>
5. 4x4 Keypad Arduino Code, Pinout & Interfacing Complete Guide
<https://robosans.com/learn/embedded/arduino/4x4-keypad-arduino-code-pinout-interfacing-lcd/>

EXPERIMENT – 2 (GO FURTHER – 1)

PROJECT NAME

BODMAS Method Based Calculator

OBJECTIVE

To design a calculator using Arduino UNO, OLED display, and keypad that performs arithmetic calculations according to the BODMAS rule.

COMPONENTS USED

Component	Quantity
Arduino UNO	1
SSD1306 OLED Display	1
5×4 Matrix Keypad	1
Breadboard	1
Jumper Wires	Required

PIN CONNECTIONS

OLED

OLED Pin	Arduino Pin
SDA	A4
SCL	A5

Keypad

Keypad Pin	Arduino Pin
R1–R5	D2–D6
C1–C4	D7–D10

KEYPAD FUNCTIONS

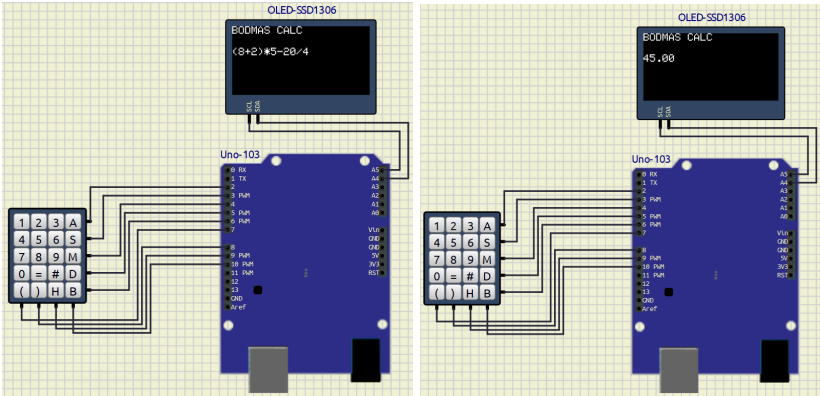
Key	Function
A	+

Key	Function
S	-
M	*
D	/
=	Calculate
#	Clear
B	Backspace
H	History

SOFTWARE USED

- Arduino IDE
- SimulIDE

CIRCUIT DIAGRAM



MAIN ARDUINO CODE

```
//=====
// BODMAS Calculator using Arduino UNO
// OLED Display + 5x4 Keypad
//
// Features:
// - Addition
// - Subtraction
// - Multiplication
// - Division
// - Brackets
// - History
// - Backspace
// - BODMAS Rule
//=====
```

```
//===== LIBRARIES =====

// I2C Communication Library
#include <Wire.h>

// OLED Graphics Library
#include <Adafruit_GFX.h>

// OLED SSD1306 Library
#include <Adafruit_SSD1306.h>

// Matrix Keypad Library
#include <Keypad.h>

//===== OLED SETTINGS =====

// OLED width
#define SCREEN_WIDTH 128

// OLED height
#define SCREEN_HEIGHT 64

// Create OLED display object
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

//===== KEYPAD SETTINGS =====

// Number of rows in keypad
const byte ROWS = 5;

// Number of columns in keypad
const byte COLS = 4;

// Keypad button layout
char keys[ROWS][COLS] = {

  {'1','2','3','A'},
  {'4','5','6','S'},
  {'7','8','9','M'},
  {'0','=',' ','D'},
  {'(',' )','H','B'}
};

// Arduino pins connected to keypad rows
byte rowPins[ROWS] = {2,3,4,5,6};

// Arduino pins connected to keypad columns
byte colPins[COLS] = {7,8,9,10};
```

```
// Create keypad object
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

//===== VARIABLES =====

// Stores current mathematical expression
String expression = "";

// Pointer used by parser
const char* expr;

// Stores last 5 calculations
String history[5];

// Current history index
int historyIndex = 0;

//===== OLED DISPLAY FUNCTION =====

// Function to refresh OLED screen
void refreshDisplay() {

    // Clear OLED screen
    display.clearDisplay();

    // Set text size
    display.setTextSize(1);

    // Set text color
    display.setTextColor(SSD1306_WHITE);

    // Title
    display.setCursor(0,0);
    display.print("BODMAS CALC");

    // Expression display position
    display.setCursor(0,20);

    // Display current expression
    display.print(expression);

    // Small delay for stable Proteus simulation
    delay(50);

    // Show everything on OLED
    display.display();
}

//===== PARSER FUNCTIONS =====
```

```
// Skip blank spaces
void skipSpaces() {

    while(*expr == ' ')
        expr++;
}

// Forward declaration
double parseExpression();

//===== NUMBER PARSER =====

// Reads numbers from expression
double parseNumber() {

    double number = 0;

    // Continue while digit found
    while(isdigit(*expr)) {

        // Build number digit by digit
        number = number * 10 + (*expr - '0');

        expr++;
    }

    return number;
}

//===== FACTOR PARSER =====

// Handles brackets
double parseFactor() {

    // Ignore spaces
    skipSpaces();

    // Check for opening bracket
    if(*expr == '(') {

        // Move to next character
        expr++;

        // Solve expression inside bracket
        double result = parseExpression();

        // Skip closing bracket
        if(*expr == ')')
            expr++;
    }
}
```

```
        return result;
    }

    // Otherwise parse normal number
    return parseNumber();
}

//===== TERM PARSER =====

// Handles multiplication and division
double parseTerm() {

    // Read first factor
    double result = parseFactor();

    while(true) {

        skipSpaces();

        // Multiplication
        if(*expr == '*') {

            expr++;

            result *= parseFactor();
        }

        // Division
        else if(*expr == '/') {

            expr++;

            result /= parseFactor();
        }

        else {
            break;
        }
    }

    return result;
}

//===== EXPRESSION PARSER =====

// Handles addition and subtraction
double parseExpression() {

    // Read first term
    double result = parseTerm();

    while(true) {
```



```
    skipSpaces();

    // Addition
    if(*expr == '+') {

        expr++;

        result += parseTerm();
    }

    // Subtraction
    else if(*expr == '-') {

        expr++;

        result -= parseTerm();
    }

    else {
        break;
    }
}

return result;
}

//===== EVALUATE FUNCTION =====

// Evaluates full mathematical expression
double evaluate(String s) {

    // Convert String to character pointer
    expr = s.c_str();

    // Start parsing expression
    return parseExpression();
}

//===== SETUP =====

void setup() {

    // Start serial monitor
    Serial.begin(9600);

    // Initialize OLED
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

    // Clear OLED
    display.clearDisplay();
}
```

```
// Text size
display.setTextSize(1);

// Text color
display.setTextColor(SSD1306_WHITE);

// Cursor position
display.setCursor(0,20);

// Startup message
display.print("READY");

// Show message
display.display();

// Wait 2 seconds
delay(2000);

// Show empty calculator screen
refreshDisplay();
}

//===== MAIN LOOP =====

void loop() {

    // Read keypad button
    char key = keypad.getKey();

    // If any key pressed
    if(key) {

        // Print pressed key on serial monitor
        Serial.print("Key: ");
        Serial.println(key);

        //===== CALCULATE =====

        // If "=" pressed
        if(key == '=') {

            // Evaluate expression
            double result = evaluate(expression);

            // Save calculation in history
            history[historyIndex] =
            expression + "=" + String(result);

            // Move to next history slot
            historyIndex++;

            // Reset history index
```

```
if(historyIndex >= 5) {

    historyIndex = 0;
}

// Replace expression with result
expression = String(result);

// Update OLED
refreshDisplay();
}

//===== CLEAR SCREEN =====

// If "#" pressed
else if(key == '#') {

    // Clear expression
    expression = "";

    // Refresh OLED
    refreshDisplay();
}

//===== BACKSPACE =====

// If "B" pressed
else if(key == 'B') {

    // Remove last character
    if(expression.length() > 0) {

        expression.remove(expression.length()-1);
    }

    // Refresh OLED
    refreshDisplay();
}

//===== HISTORY =====

// If "H" pressed
else if(key == 'H') {

    // Clear OLED
    display.clearDisplay();

    // OLED settings
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
```

```
// Title
display.setCursor(0,0);
display.print("HISTORY");

int y = 12;

// Show saved history
for(int i=0; i<5; i++) {

    if(history[i] != "") {

        display.setCursor(0,y);

        display.print(history[i]);

        y += 10;
    }
}

// Show on OLED
display.display();

// Wait 4 seconds
delay(4000);

// Return to calculator screen
refreshDisplay();
}

//===== NORMAL INPUT =====

else {

    // Convert keypad symbols

    if(key == 'A')
        expression += '+';

    else if(key == 'S')
        expression += '-';

    else if(key == 'M')
        expression += '*';

    else if(key == 'D')
        expression += '/';

    // Add normal characters
    else
        expression += key;

    // Refresh OLED
    refreshDisplay();
}
```

```
}  
}  
}
```

WORKING PRINCIPLE

1. User enters expression using keypad.
 2. Arduino receives keypad input.
 3. OLED displays the expression.
 4. On pressing `=`, Arduino evaluates the expression using BODMAS rule.
 5. Result is displayed on OLED.
 6. Previous calculations are stored in history.
-

OUTPUT / RESULT

Example

Input:

```
(8+2)*5-20/4
```

Output:

```
45
```

The calculator successfully performed calculations according to the BODMAS rule.

ADVANTAGES

- Supports BODMAS calculations
 - Compact and portable
 - Real-time OLED display
 - History and backspace support
-

APPLICATIONS

- Educational projects
 - Scientific calculator systems
 - Arduino learning
-

CONCLUSION

The BODMAS calculator using Arduino UNO was successfully implemented. The calculator correctly evaluated mathematical expressions and displayed the results on the OLED display.

REFERENCES

1. <https://www.arduino.cc/>
2. <https://docs.arduino.cc/hardware/uno-rev3/>
3. https://github.com/adafruit/Adafruit_SSD1306
4. <https://playground.arduino.cc/Code/Keypad/>
5. <https://www.circuitbasics.com/how-to-set-up-a-keypad-on-an-arduino/>